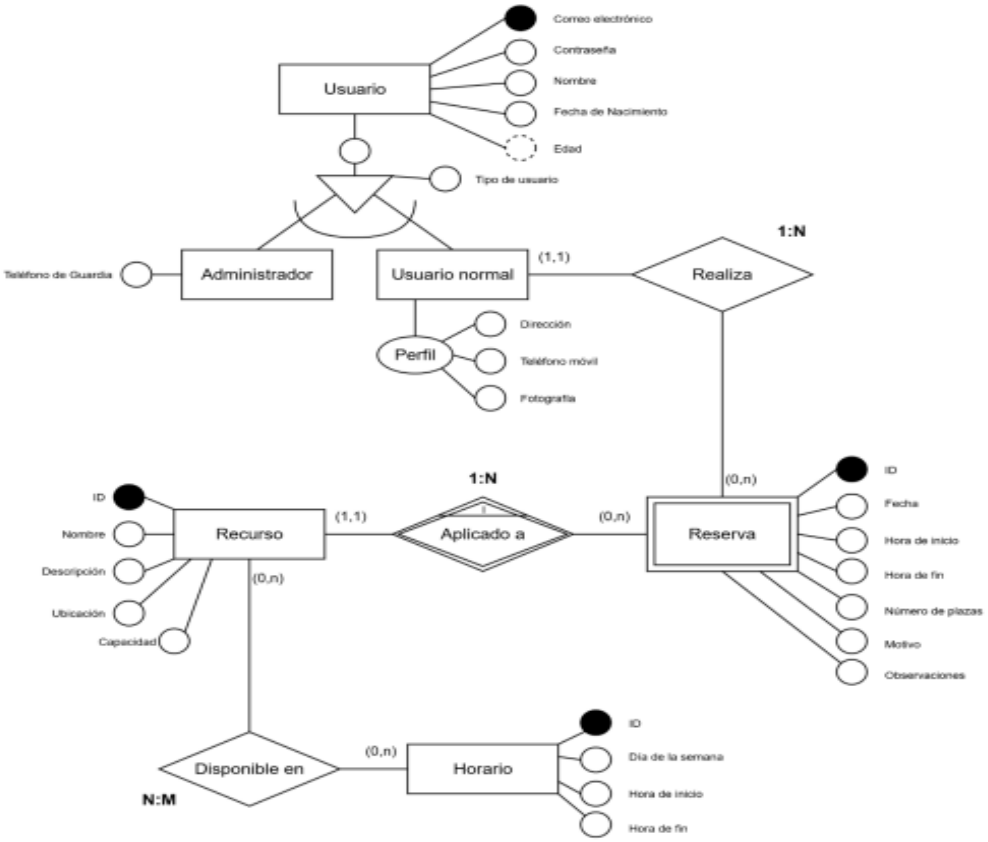
CRUD en la DB Sistema de Reservas: Proyecto Intermodular

Introducción

En este tema vamos a realizar una práctica relacionada con el proyecto intermodular en el que tendrás que crear una aplicación en Java que permita realizar el mantenimiento de parte o toda la aplicación del proyecto de Reservas.

Si no estás matriculado en base de datos y/o no estás matriculado en el proyecto intermodular, tendrás que indicármelo para que te pase la base de datos propuesta y desarrolles el programa sobre esta base de datos.

La base de datos utilizada corresponde a un **sistema de reservas de recursos**, en el que existen diferentes entidades como usuarios, recursos, horarios y reservas. Su modelo E/R es



Comentarios

ID de reserva se repite por cada recurso (Recurso A: 1, 2... Recurso B: 1, 2...).

El horario representa la disponibilidad recurrente de los recursos, como disponible los lunes de 9:00 a 10:00 por ejemplo

Recordad que no hay una única solución de esquema. Tomaremos esta para que la dificultad sea la misma para todos.

El objetivo es desarrollar una **aplicación Java de consola conectada a la base de datos mediante JDBC** que permita realizar operaciones de mantenimiento sobre las tablas de la base de datos.

Estas operaciones consisten en las operaciones **CRUD**:

- **C**reate → insertar registros
- **R**ead → consultar registros
- **U**pdate → modificar registros
- **D**elete → eliminar registros

Cada alumno podrá decidir **hasta qué nivel implementar la aplicación**, lo cual determinará la puntuación máxima que podrá obtener.

Objetivos de la tarea

El objetivo principal es que el alumnado:

- Aprenda a conectar Java con una base de datos mediante **JDBC**
- Implemente operaciones **CRUD**
- Diseñe una aplicación modular
- Utilice buenas prácticas de programación
- Aplique el patrón **Modelo-Vista-Controlador (MVC)**

Base de datos

Si habéis llevado el proyecto intermodular, tendréis una base de datos en **MySql/MariaDB** llamada

sistema_reservas

Esta base de datos incluye las siguientes tablas principales(podrían ser otros en tu diseño):

USUARIO

ADMINISTRADOR

USUARIONORMAL

RECURSO

HORARIO

DISPONIBLEEN

RESERVA

Funcionalidades a implementar

La aplicación deberá permitir realizar operaciones de mantenimiento sobre las tablas de la base de datos. La operaciones sobre sobre las tablas serán:

- insertar registros
- modificar registros
- eliminar registros
- consultar registros (todos los registros, varios listados por diferentes campos…)

Las anteriores operaciones se pueden realizar sobre:

- gestionar usuarios
  - gestión administrador
  - gestión usuario_normal
- gestionar recursos
- gestionar horarios
- gestionar reservas

Opciones de desarrollo y puntuación

Los alumnos podrán elegir **hasta qué nivel desarrollar la aplicación**.

Nivel	Descripción	Nota máxima
Nivel 1	CRUD de una sola tabla **SIN** utilizar MVC Ejemplo	6
Nivel 2	CRUD de una tabla utilizando patrón MVC Ejemplo	7
Nivel 3	CRUD de dos tablas relacionadas utilizando MVC Ejemplo	8
Nivel 4	CRUD de tres tablas relacionadas utilizando MVC	9
Nivel 5	CRUD de toda la base de datos utilizando el MVC	10

Nivel mínimo para aprobar

El nivel mínimo para aprobar consiste en:

- Implementar el CRUD de una tabla
- Con conexión JDBC funcional
- Aplicación ejecutable desde consola

Este nivel permitirá obtener hasta un 6. Los siguientes niveles exigen implementar el patrón MVC sobre una o más tablas

Requisitos técnicos

La aplicación deberá cumplir los siguientes requisitos:

- Lenguaje: **Java**
- Conexión a base de datos mediante **JDBC a MySql/MariaDB**
- Aplicación de **consola**
- Manejo adecuado de **excepciones**
- Código organizado en paquetes y clases
- Documentación en código y JavaDoc
- Patrón **Modelo-Vista-Controlador (MVC) si opta por nivel mayor de 1**

Estructura recomendada del proyecto

La estructura recomendada para el model MVC es:

```
src
 ├ app
 |   └ Main.java
 |
 ├ model
 |   ├ Usuario.java
 |   ├ Administrador.java
 |   ├ UsuarioNormal.java
 |   ├ Recurso.java
 |   ├ Horario.java
 |   ├ DisponibleEn.java
 |   └ Reserva.java
 |
 ├ dao
 |   ├ DBConnection.java
 |   ├ UsuarioDAO.java
 |   ├ AdministradorDAO.java
 |   ├ UsuarioNormalDAO.java
 |   ├ RecursoDAO.java
 |   ├ HorarioDAO.java
 |   ├ DisponibleEnDAO.java
 |   └ ReservaDAO.java
 |
 ├ view
 |   ├ ConsolaView.java
 |   ├ UsuarioView.java
 |   ├ RecursoView.java
 |   ├ HorarioView.java
 |   └ ReservaView.java
 |
 └ controller
     ├ AppController.java
     ├ UsuarioController.java
     ├ RecursoController.java
     ├ HorarioController.java
     └ ReservaController.java
```

Funcionalidades mínimas del CRUD

Para cada tabla se deberá implementar al menos:

Insertar

INSERT INTO tabla (...)

Modificar

UPDATE tabla SET ...

Eliminar

DELETE FROM tabla

Consultas (al menos 2)

Selección de todos

Selección por alguna condición (buscar por nombre, buscar por fecha, totales….)

Menú principal recomendado

Se recomienda que la aplicación comience con el menú con todas las opciones y poco a poco se desarrolle cada opción (las que no se desarrollen que muestren "en desarrollo…" cuando se seleccione)

```
===== SISTEMA DE RESERVAS =====

1. Gestionar usuarios
2. Gestionar recursos
3. Gestionar horarios
4. Gestionar reservas

0. Salir
```

Un menú de la aplicación completa podría ser el siguiente. No tiene por qué ser el mismo:

```
Iniciar aplicación
└── Menú principal
    ├── 1. Gestión de usuarios
    │   ├── Alta usuario
    │   ├── Baja usuario
    │   ├── Modificar usuario
    │   ├── Listar usuarios
    │   ├── Buscar por nombre
    │   ├── Buscar por correo
    │   └── Volver
    │
    ├── 2. Gestión de recursos
    │   ├── Alta recurso
    │   ├── Baja recurso
    │   ├── Modificar recurso
    │   ├── Listar recursos
    │   ├── Buscar recurso por nombre
    │   └── Volver
    │
    ├── 3. Gestión de horarios
    │   ├── Alta horario
    │   ├── Baja horario
    │   ├── Modificar horario
    │   ├── Listar horarios
    │   └── Volver
    │
    ├── 4. Gestión de reservas
    │   ├── Alta reserva
    │   ├── Baja reserva
    │   ├── Modificar reserva
    │   ├── Listar reservas
    │   ├── Buscar reservas por usuario
    │   ├── Buscar reservas por recurso
    │   └── Volver
    │
    ├── 5. Consultas y listados
    │   ├── Listar administradores
    │   ├── Listar usuarios normales
    │   ├── Listar recursos disponibles por horario
    │   ├── Listar reservas de un usuario
    │   ├── Listar reservas de un recurso
    │   ├── Resumen general
    │   └── Volver
    │
    └── 0. Salir
```

Recomendación de desarrollo según dificultad

Se recomienda comenzar por los mantenimientos de las tablas más simples sin dependencias de relación (recurso, usuario) y continuar con las tablas con las relaciones dependientes (horario, reserva...).

Dependiendo del nivel elegido se implementarán solo algunas tablas.

Ejemplo:

Nivel	Tablas sugeridas
1	RECURSO (la más sencilla) o USUARIO
2	RECURSO (la más sencilla) o USUARIO
3	USUARIO + RESERVA
4	USUARIO + RECURSO + RESERVA
5	Todas las tablas

Entrega

- El proyecto estará en GitHub en un repositorio privado al que darás permiso al usuario:
  iessochoalmj201617@gmail.com

  Incluirá el proyecto Java y el script sql de la base de datos con algunos ejemplos de datos cargados.
  Entrega del enlace al repositorio

- Vídeo Explicativo (**URL a OneDrive**): Duración recomendada: 3 a 5 minutos en que:
  - Explicación del proyecto (código, estructura, sentencias SQL utilizadas…). **Podría convocarse al alumno en caso de duda de realización del proyecto por parte del alumno**
  - Funcionamiento del programa donde muestras todas las opciones de menú del programa (CRUD)
  - Conclusión final: dificultades, aprendizaje, ampliaciones pendientes…

Rúbrica de evaluación

Esta práctica corresponde al RA:

RA9. Gestiona información almacenada en bases de datos manteniendo la integridad y consistencia de los datos.

Criterios de evaluación

b) Se han programado conexiones con bases de datos.

c) Se ha escrito un código para almacenar información en bases de datos.

d) Se han creado programas para recuperar y mostrar información almacenada en bases de datos.

e) Se han efectuado borrados y modificaciones sobre la información almacenada.

f) Se han creado aplicaciones que muestren la información almacenada en bases de datos.

g) Se han creado aplicaciones para gestionar la información presente en bases de datos.

La evaluación del RA completo se hará según la siguiente rúbrica.

Criterio	Descripción	Puntuación
Funcionamiento de la aplicación	La aplicación ejecuta correctamente las operaciones CRUD	4
Uso del patrón MVC. Si se opta por esta opción. En caso de que no, estos dos puntos irán al primer apartado	Separación correcta entre modelo, vista y controlador.	2
Estructura del código	Código organizado en clases y métodos claros	1
Nombrado de variables	Uso de nombres claros y significativos	1
Documentación	Uso correcto de JavaDoc y comentarios	1
Manejo de errores	Control adecuado de excepciones	1

Esta rúbrica será ponderada según la opción de dificultad elegida por el alumno

Nivel	Descripción	Nota máxima
Nivel 1	CRUD de una sola tabla **SIN** utilizar MVC	6

Nivel	Descripción	Nota máxima
	Ejemplo	
Nivel 2	CRUD de una tabla utilizando patrón MVC Ejemplo	7
Nivel 3	CRUD de dos tablas relacionadas utilizando MVC Ejemplo	8
Nivel 4	CRUD de tres tablas relacionadas utilizando MVC	9
Nivel 5	CRUD de toda la base de datos utilizando el MVC	10

La nota del RA9 será:

Nota Práctica *0,4 + Nota Examen *0,6